

A-MORE

Team Guide — Status, Code Walkthrough & How to Run It

Analytics for Meaningful Online Romantic Encounters

Repository	github.com/Shirbat12/Amore (branch main)
Local path	C:\Users\shirb\OneDrive\Desktop\Amore
Team	Shir (data-science core), Michal (backend/NLP/db), Liel (front-end), Mai (docs/pilot)
Status	Working skeleton, end-to-end verified (9/9 tests pass; prediction $p=0.87$ on synthetic data)

Contents

1. What A-MORE Is (60-second version)
2. How the Code Is Organized
3. How It Works — a request walkthrough
4. What's Done
5. What's Left To Do
6. How the Team Works Together (git)
7. How To Run It — End To End ← start here
8. Troubleshooting

1. What A-MORE Is

A-MORE is a decision-support layer over dating apps. Instead of chasing more matches, it closes the loop between the dry digital profile and what actually happened on the date. After each date the user fills a short VAS “Vibe” questionnaire; a language model turns the free text into a fixed list of personality tags; a per-user statistical engine learns which profile features really predict a good date; and a browser extension paints a live match score on new profiles.

```
Profile (DOM)  -> Match-score overlay  -> Real date  -> Vibe questionnaire
      ^                                     |
      |                                     v
Dashboard insights <- Per-user model update <- NLP tag extraction
```

Three tiers: **client** (Chrome extension, questionnaire, dashboard), **server** (FastAPI + the 4-module pipeline), **storage** (SQLite + JSON document store). The whole thing runs locally with no API key — the NLP step has a deterministic offline fallback.

2. How the Code Is Organized

Every folder maps to one responsibility. Comments in the code carry §2.3.x tags that point back to the project book.

Folder	What lives there
server/	The backend. FastAPI app (app.py) + the API routes.
server/pipeline/	The 4 brain modules: quantify_experience (2.3.1), revealed_preferences (2.3.2), predictor (2.3.3), presentation (2.3.4).
server/nlp/	Gemini client (+ offline mock), the closed tag list, and the prompt.
server/models/	Shared data types: DateRecord, VibeForm, TagOutput, User.
server/db/	Persistence: relational.py (SQLite) + document_store.py (JSON).
extension/	Chrome MV3 extension: scraper, score overlay, service worker, popup.
questionnaire/	The post-date Vibe form (HTML/JS) that posts to /feedback.
dashboard/	The insights dashboard: 4 Chart.js visualizations.
evaluation/	KPIs, ablation experiments, and the pilot protocol.
data/	The tag taxonomy + a synthetic-data generator for development.
tests/	Pytest suite covering the pipeline modules.

3. How It Works — a request walkthrough

Two flows matter. Read these once and the codebase clicks.

A) Submitting a date (POST /feedback)

- The **questionnaire** sends a VibeForm (sliders + word-bank tags + free text).
- `routes_feedback.py` calls **quantify_experience()** — the free text goes to the Gemini client (or the offline mock), which returns closed-list tags + sentiment.
- A `DateRecord` is built and saved to both stores (SQLite + JSON).

B) Scoring a new profile (POST /score)

- The **extension** scrapes dry profile tokens and sends them with the user id.
- `routes_score.py` loads the user's history and calls **presentation.build_overlay()**.
- Inside, **predictor.score_profile()** fits the per-user model: a population prior if there are fewer than `MIN_DATES` dates, otherwise Ridge regression; it also pulls the top Spearman correlations to explain the score.
- The extension paints the returned `{score, confidence, reasons}` as an overlay.

The dashboard (GET /insights/<user>) runs **build_dashboard()**, which packages the correlation heatmap, the date funnel, a leave-one-out predicted-vs-actual scatter, per-tag boxplots, and natural-language insights.

Key idea: the model is **per-user and explainable** by design — the language model only encodes text into tags; the actual prediction is transparent statistics (Spearman + Ridge), not a black box.

4. What's Done

The full skeleton is built and verified working end-to-end. **Done** = implemented and tested; **Partial** = works but with a placeholder to finish.

Component	What's there	Status
Backend API	app.py + /score, /feedback, /insights routes; CORS open for dev	Done
Pipeline 2.3.1	quantify_experience: NLP tags + sentiment -> DateRecord, retry + graceful fallback	Done
Pipeline 2.3.2	revealed_preferences: feature->trait & feature->VAS tables	Done
Pipeline 2.3.3	predictor: Spearman+FDR explanation, RidgeCV, cold-start prior, scoring	Done
Pipeline 2.3.4	presentation: overlay + dashboard payloads (4 visualizations)	Done
NLP	closed 19-tag list, prompt + few-shot, Gemini client	Partial
Database	SQLite relational store + JSON document store, lazy init	Done
Models	DateRecord, VibeForm, TagOutput, User (pydantic)	Done
Extension	MV3 manifest, overlay, service worker, popup settings	Partial
Questionnaire	RTL Hebrew Vibe form posting to /feedback	Done
Dashboard	4 Chart.js charts + insights, reads /insights	Done
Evaluation	KPIs (conversion, SUS, response rate, accuracy), ablation, protocol	Done
Data	tag taxonomy CSV + synthetic generator + 30 sample dates	Done
Tests	9 pytest tests across the pipeline — all passing	Done

Verified: `pytest` → 9 passed; a full `/feedback` → `/score` → `/insights` smoke test recovered the planted signal (good profile 83.7 vs bad 30.4; prediction $p = 0.87$).

5. What's Left To Do

Ordered roughly by priority. The two “Partial” items above are the first real tasks.

Pri	Task	Detail	Owner
P1	Real DOM selectors	extension/content/dom_scraper.js has placeholder selectors. Inspect the live Tinder/Hinge DOM and fill them in so real profiles get scraped.	Liel
P1	Live Gemini	Add GEMINI_API_KEY in .env and validate that real extraction matches the closed-tag schema; tune prompts.py.	Michal
P1	Phone-exchange trigger	questionnaire/triggers.md is a spec only. Implement / simulate the trigger that sends the Vibe form.	Michal/Mai
P2	Selection logging	The Decision-Alignment KPI needs the extension to log which profiles the user accepts. Not captured yet.	Liel
P2	FastAPI lifespan	app.py uses the deprecated @on_event('startup'); migrate to the lifespan handler.	Michal
P2	More tests	Add API-route tests, presentation tests, and KPI edge cases.	All
P3	Predictor caching	Model is refit per request; cache per user and invalidate on new feedback.	Shir
P3	True boxplots	Dashboard boxplots are a min/mean/max approximation; use a real boxplot plugin if desired.	Liel
P3	Hardening	Tighten CORS, add auth/multi-user, input validation, rate limiting before any real users.	Michal
P3	Pilot + deploy	Run the 50-user / 4-week pilot (evaluation/experiment_protocol.md); host the backend; package the extension.	Mai/All

6. How the Team Works Together (git)

The code is on main. To avoid stepping on each other, work on a branch and open a Pull Request. Each person mostly owns different folders (Section 2), so conflicts should be rare.

First time — get the code

```
git clone https://github.com/Shirbat12/Amore.git
cd Amore
```

Every time you start a task

```
git checkout main
git pull                # get everyone's latest work
git checkout -b feat/my-task # your own branch
# ... edit code ...
git add -A
git commit -m "clear message about what you did"
git push -u origin feat/my-task
# then open a Pull Request on GitHub and ask a teammate to review
```

Rules of thumb

- Never commit the `.data/` folder or `.env` (already git-ignored).
- Pull main before branching so you start from the latest.
- Keep commits small with clear messages — it makes review and the project book easier.
- Run `pytest` before pushing backend changes.

7. How To Run It — End To End

Follow these in order on Windows (PowerShell). After this you'll have the API running, data flowing, and all three front-ends working. Commands assume you are in the repo root (`...\Desktop\Amore`).

Step 0 — Prerequisites

- Python 3.12 (on Shir's machine the working interpreter is `C:\Users\shirb\AppData\Local\Programs\Python\Python312\python.exe` — the bare `python` command is a broken Store stub).
- Git, and Google Chrome (for the extension).

Step 1 — Get the code

```
git clone https://github.com/Shirbat12/Amore.git
cd Amore
```

Step 2 — Create a virtual env & install

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

If python isn't found, use the full path from Step 0 instead of `python`.

Step 3 — Seed sample data (so charts have something to show)

```
python -m data.samples.generate_samples
```

Step 4 — Run the backend

```
uvicorn server.app:app --reload
```

Leave this running. Open `http://localhost:8000/docs` to see and try every endpoint. Load the 30 sample dates into a user with this one-off (new terminal, venv active):

```
python -c "from server.db import relational as r; from data.samples.generate_samples import generate;
[r.save_date(d) for d in generate(n=30, user_id='demo_user')]; print('seeded demo_user')"
```

Step 5 — Open the dashboard

Serve the repo statically (so the browser can load the files), then open the dashboard pointed at the API and the seeded user:

```
python -m http.server 5500
# then in the browser:
http://localhost:5500/dashboard/index.html?api=http://localhost:8000&user=demo_user
```

Step 6 — Submit a real date through the questionnaire

```
http://localhost:5500/questionnaire/vibe_form.html?user=demo_user&profile=interest:travel,hobby:cooking
```

Fill the sliders, pick tags, write a sentence (e.g. "■■■■ ■■■■, ■■■■ ■■■■ ■■■■■■ ■■■■■■"), submit — it saves and shows the extracted tags. Refresh the dashboard to see it.

Step 7 — Load the Chrome extension

- Go to `chrome://extensions`, turn on **Developer mode**.
- Click **Load unpacked** and select the extension/ folder.
- Click the A-MORE icon → set API base `http://localhost:8000` and user `demo_user` → Save.

- Note: the overlay needs real profile selectors (To-Do P1) to light up on Tinder/Hinge; the score endpoint itself already works (try it from /docs).

Step 8 — Run the tests

```
pytest -q
```

Expected: 9 passed.

8. Troubleshooting

Symptom	Fix
'python' is not found / opens the Store	Use the full interpreter path from Step 0, or disable the Windows 'App execution alias' for Python.
UnicodeEncodeError when printing Hebrew	Run <code>set PYTHONIOENCODING=utf-8</code> before the command. The script usually still succeeded — it's only the console print that failed.
Dashboard/questionnaire can't reach the API	Make sure uvicorn is running (Step 4) and you opened the pages via <code>http://localhost:5500</code> (not <code>file://</code>). CORS is open in dev.
Extension overlay doesn't appear	Expected until the DOM selectors (To-Do P1) are filled. The <code>/score</code> endpoint works regardless — verify via <code>/docs</code> .
no such table / empty results	The DB auto-creates on first use. Seed data with Step 3 + the Step 4 one-liner.
Gemini not used	Without <code>GEMINI_API_KEY</code> the offline mock tagger runs by design. Add the key in <code>.env</code> to use the real model.

Questions about a specific module? Open its file — each one starts with a docstring that explains its role and cites the matching project-book section (§2.3.x). Start with `server/app.py` and follow the imports.